

Filtering I

Honza Černocký, ÚPGM

Please open Python notebook `filtering_1`

Agenda

- **Terminology and motivation**
- Basic operations needed for filtering
- A simple FIR filter
- Convolution (and needed signal manipulations)
- Filters with feedback - IIR
- Frequency response - an introduction
- Summary

Aliases

- Numerical filters (číslicové filtry)
- Discrete systems (diskrétní systémy)
- Discrete-time systems (systémy s diskrétním časem)
- etc.

What for I – Selecting some parts of spectrum

Demos on 5 secs of rock music (who knows the band ?)

- **#demo_lowpass** – max frequency 500 Hz – **low-pass filter** (dolní propust')
- **#demo_highpass** – min. frequency 5000 Hz – **high-pass filter** (horní propust')
- **#demo_terlephone** – simulating the classical telephone network from 300 to 3400 Hz. – **band-pass filter** (pásmová propust')

What for II – attenuating some nuisance components

`#demo_1kHz_cleaning` – signal polluted with a 1 kHz beep

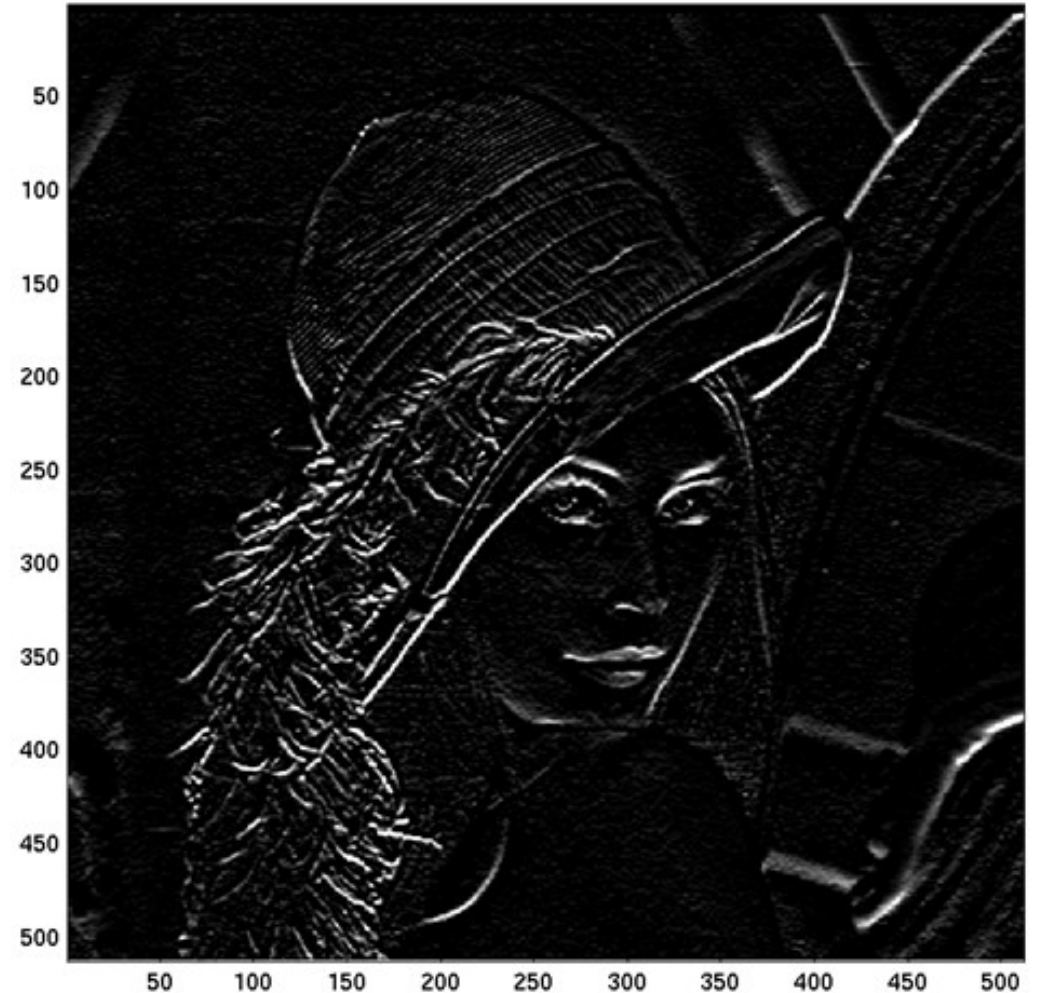
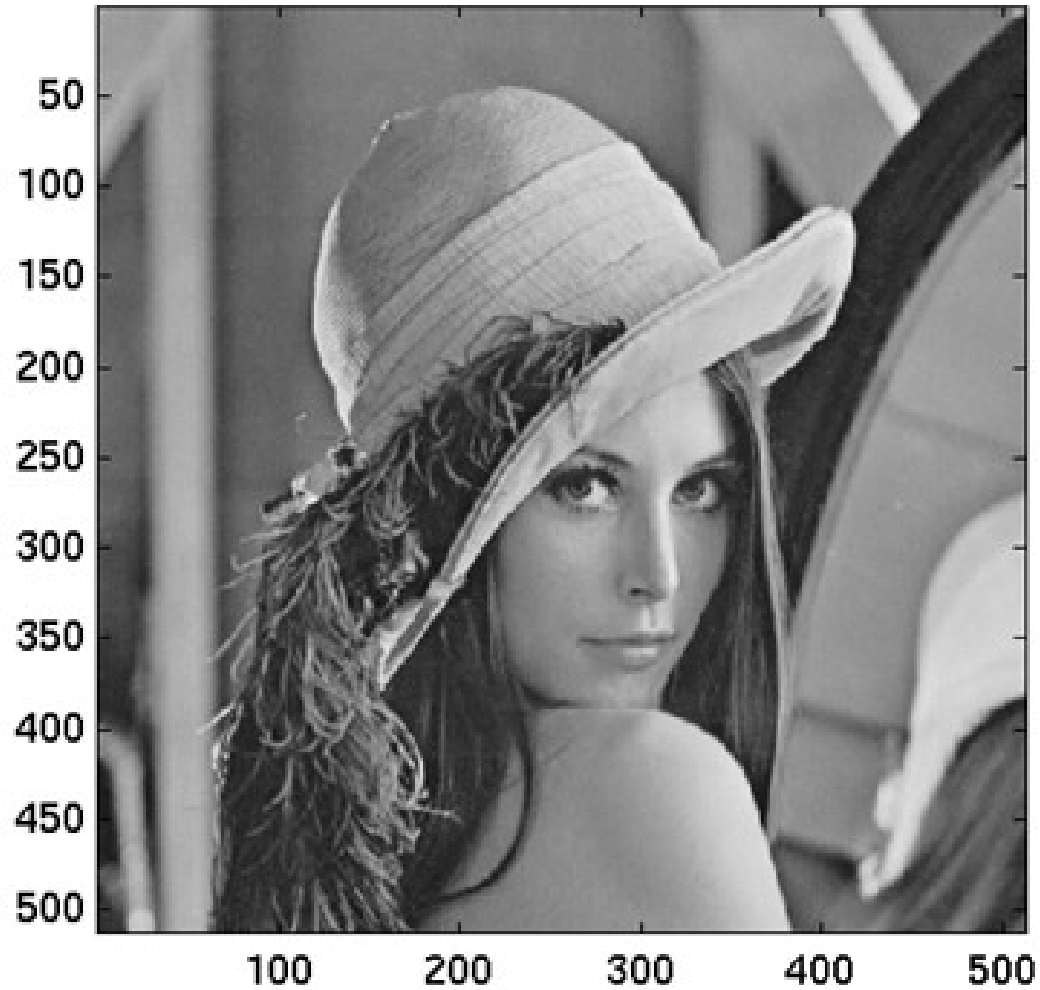
- **Band-stop filter** (`pásmová zadrž`) designed to kill 1 kHz frequency (and some around ...)

What for III – detection of signal in noise by a „matched“ filter

- When trying to transmit signals through noisy channel, might need high energy to get them through the noise ...
- Or use a matched filter that is „looking“ for the signal in the noise.

#demo_matched_filter

What for IV – images

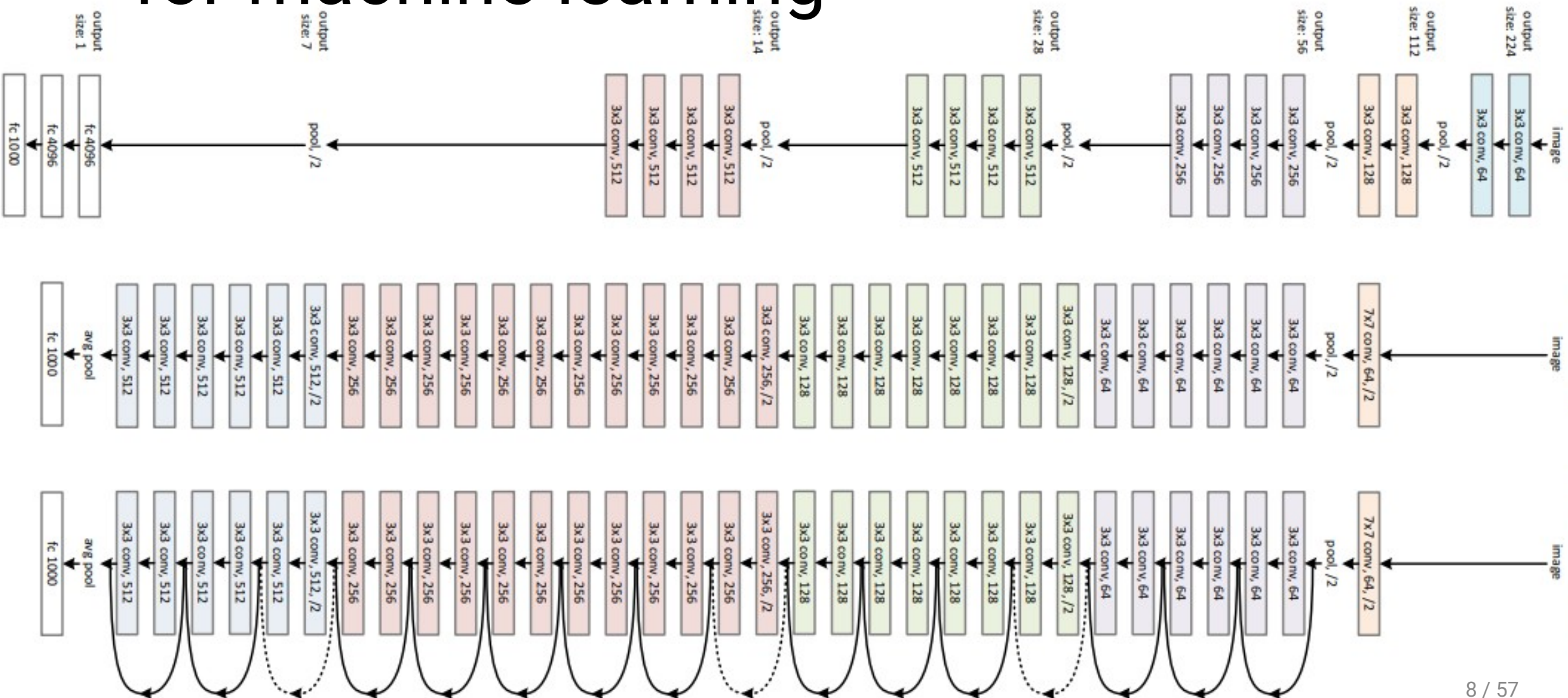


What for V. – Learned feature extraction for machine learning

VGG-19

34-layer plain

34-layer residual



Agenda

- Terminology and motivation
- **Basic operations needed for filtering**
- A simple FIR filter
- Convolution (and needed signal manipulations)
- Filters with feedback - IIR
- Frequency response - an introduction
- Summary

What do we need to implement filters in C ? [wire.c](#)

- Sampled signal
 - At some sampling frequency (most often 8kHz, 16kHz, 44.1kHz (CD), 48kHz, 96kHz)
 - For visualization, will show it as continuous ([stem vs. plot](#))
 - For ISS no special formats (MP3, OGG, WAV), but a plain RAW
 - No header, sequence of 16-bit shorts

```
#include <stdio.h>
#include <stdint.h>
#define MAXN 1000000

FILE *ifile, *ofile;
int16_t x[MAXN], y[MAXN];
uint32_t N;

main (int argc, char *argv[]) {
    ifile = fopen(argv[1], "rb");
    ofile = fopen(argv[2], "wb");

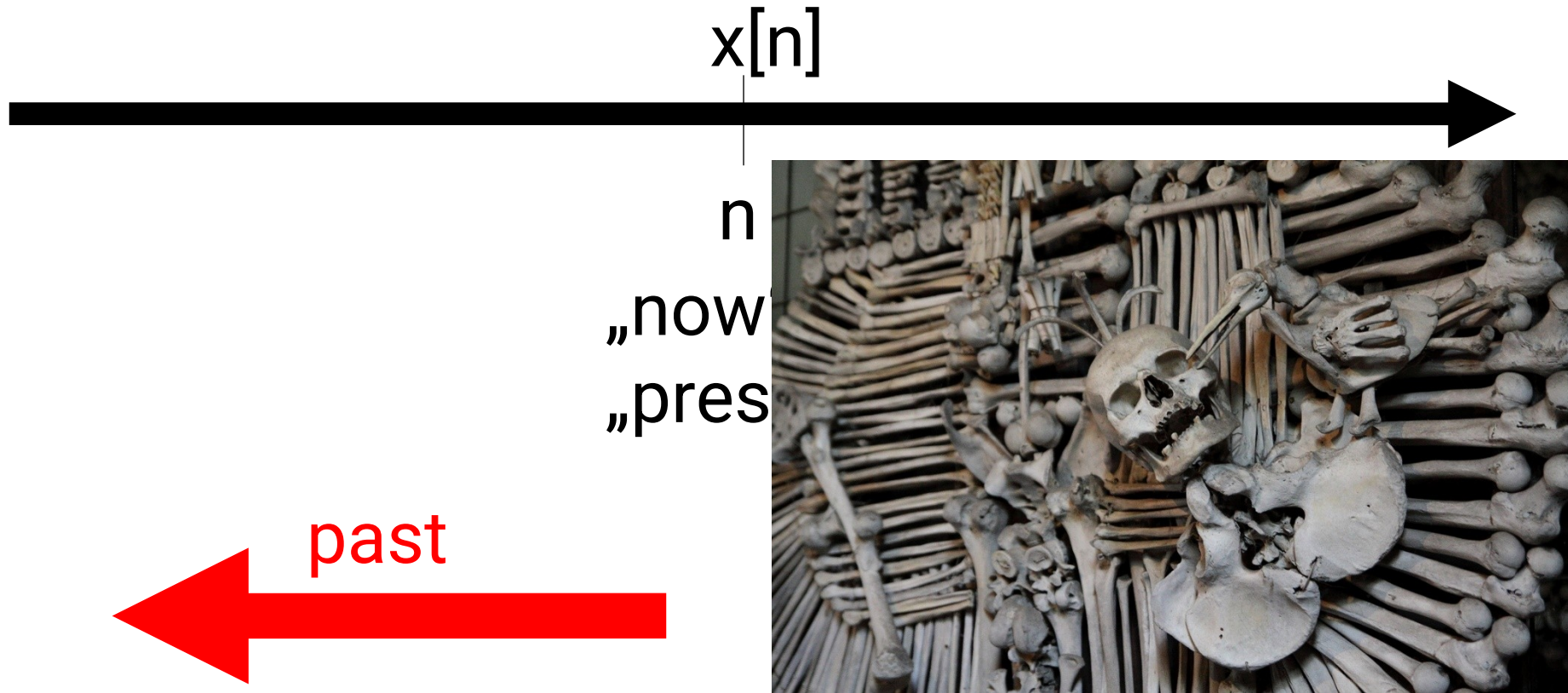
    /* read it */
    N = fread(x, sizeof(int16_t), MAXN, ifile); fclose(ifile);
    /* write it */
    fwrite(x, sizeof(int16_t), N, ofile); fclose(ofile);
}
```

3 simple operations needed for filtering

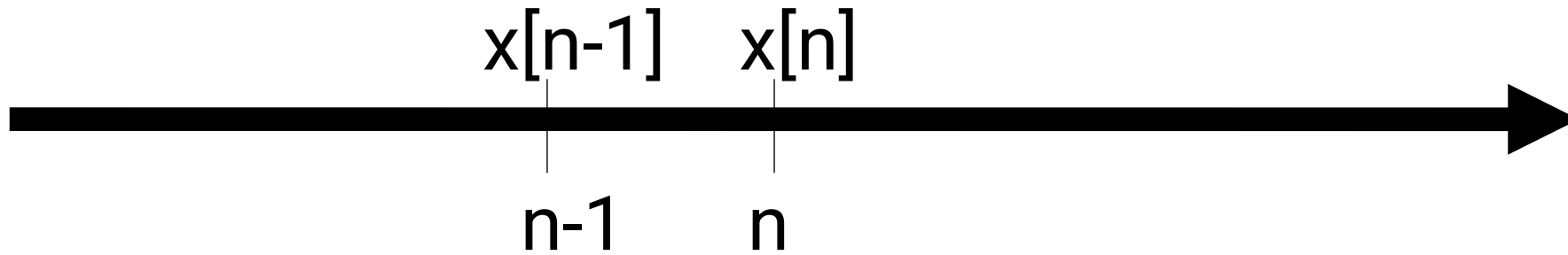
- Multiplication with a constant (násobení) *
- Summation / addition (sčítání) +
- Shift (posun, resp. zpoždění) – implemented by a memory.

Special signal processors even have instructions called MACD (multiply-accumulate-delay).

Shift ? Look only to the past !



Shift off-line



... just changing the value of the pointer.

Shift on-line

- We have only $x[n]$
- Function with a **memory** – making the past samples older (“**staření vzorků**”)

```
float filter (float xn) {  
    static float xn1, xn2;  
    ... some processing ...  
    xn2 = xn1;  
    xn1 = xn;  
    return something;  
}
```

Agenda

- Terminology and motivation
- Basic operations needed for filtering
- **A simple FIR filter**
- Convolution (and needed signal manipulations)
- Filters with feedback - IIR
- Frequency response - an introduction
- Summary

A simple filter

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] + b_3x[n-3]$$

... this is called **difference equation** (diferenční rovnice)

Attention !!! This is **NOT** differential equation (diferenciální rovnice)
– we'll see these for continuous-time systems (systémy se spojitým časem)

How does it work for sample $n=45$?

n	40	41	42	43	44	45	46	47	48	49	50
input	x[40]	x[41]	x[42]	x[43]	x[44]	x[45]	x[46]	x[47]	x[48]	x[49]	x[50]
			b3	b2	b1	b0					
			b3 * x[42]	b2 * x[43]	b1 * x[44]	b0 * x[45]					
			Σ								
output	y[40]	y[41]	y[42]	y[43]	y[44]	y[45]	y[46]	y[47]	y[48]	y[49]	y[50]

How does it work for sample $n=50$?

n	40	41	42	43	44	45	46	47	48	49	50
input	x[40]	x[41]	x[42]	x[43]	x[44]	x[45]	x[46]	x[47]	x[48]	x[49]	x[50]
								b3	b2	b1	b0
								b3 * x[47]	b2 * x[48]	b1 * x[49]	b0 * x[50]
								Σ			
output	y[40]	y[41]	y[42]	y[43]	y[44]	y[45]	y[46]	y[47]	y[48]	y[49]	y[50]

Implementation off-line `fir_offline.c`

- absolutely verbatim re-writing of the difference equation

```
for (n=3; n<N; n++) {  
    y[n] = b0 * x[n] + b1 * x[n-1] + b2 * x[n-2]  
    + b3 * x[n-3];  
}
```

... why do we begin from 3 ?

Implementation on-line `fir_online.c`

- A function for filtering is called for every sample
- It must memorize the past samples

```
float filter (float xn) {  
    static float xn1 = 0.0, xn2 = 0.0, xn3 = 0.0;  
    float yn;  
    yn = b0 * xn + b1 * xn1 + b2 * xn2 + b3 * xn3;  
    xn3 = xn2;  
    xn2 = xn1;  
    xn1 = xn;  
    return (yn);  
}
```

+ a call in main:

```
for (n=0; n<N; n++) {  
    y[n] = filter (x[n]);  
}
```

Trying our intuition – what will this filter do for:

	b0	b1	b2	b3
Filter1	0.25	0.25	0.25	0.25
Filter2	0.25	-0.25	0.25	-0.25
Filter3	1	0	0	0
Filter4	2	0	0	0
Filter5	0	0	0	1

Showing on a segment of noisy cosine ...

#filter1 ... #filter5

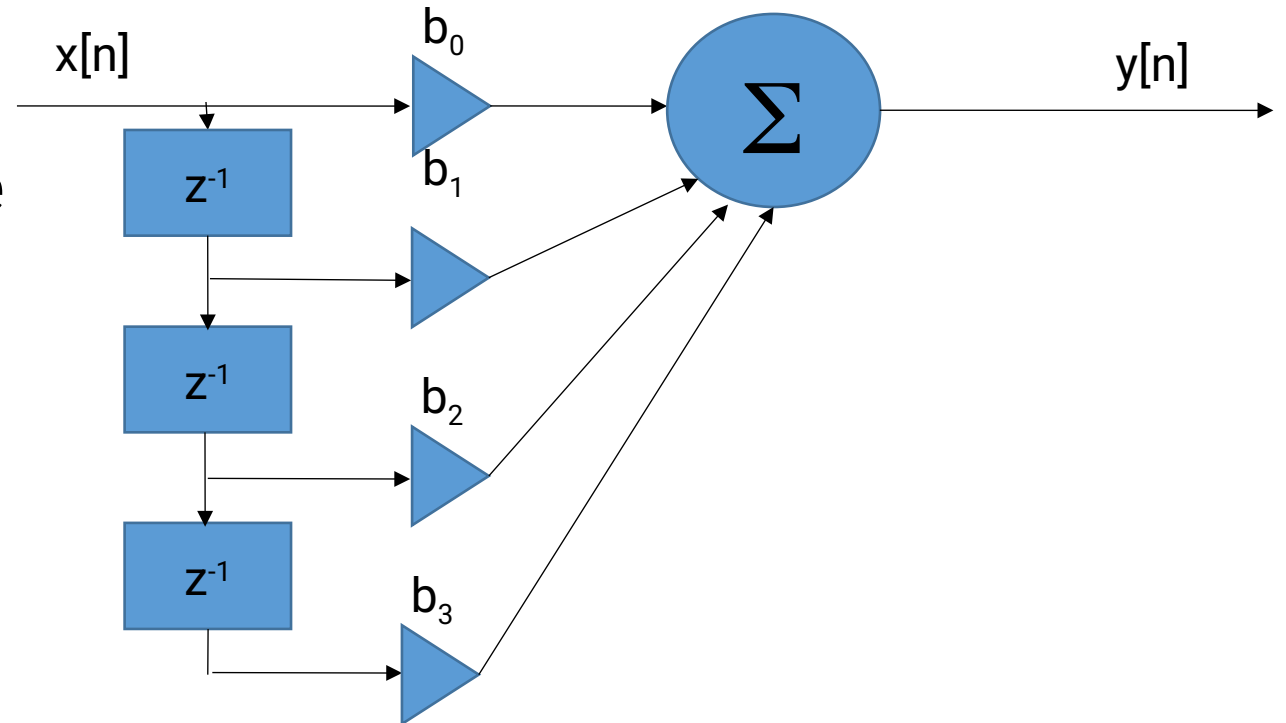
How to represent a filter ?

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] + b_3x[n-3]$$

... **difference equation** (diferenční rovnice)

scheme (schéma, diagram ...)

- z^{-1} blocks are delays by 1 sample (explanation to come later)
- Triangles are multiplications
- Big circle is summation.



Impulse response (impulsní odezva) – reaction to unit pulse

n	-4	-3	-2	-1	0	1	2	3	4	5	6
input	0	0	0	0	1	0	0	0	0	0	0
		b3	b2	b1	b0						
		b3 * 0	b2 * 0	b1 * 0	b0 * 1						
		Σ									
output					b0						

n	-4	-3	-2	-1	0	1	2	3	4	5	6
input	0	0	0	0	1	0	0	0	0	0	0
					b3	b2	b1	b0			
					b3 * 1	b2 * 0	b1 * 0	b0 * 0			
					Σ						
output								b3			

Impulse response

- the impulse response is exactly equivalent to the coefficients:
 - $h[k] = b_k$ for $k = 0, 1, 2, 3$
 - $h[k] = 0$ otherwise
- The length of this impulse response is 4, so it is finite (**konečná impulsní odezva**) – **FIR – finite impulse response filter**

... What should we do to implement a filter with an infinite impulse response (**nekonečná impulsní odezva**) ?

Agenda

- Terminology and motivation
- Basic operations needed for filtering
- A simple FIR filter
- **Convolution (and needed signal manipulations)**
- Filters with feedback - IIR
- Frequency response - an introduction
- Summary

Convolution

$$\begin{aligned} y[n] &= x[n] \star h[n] \\ &= \sum_{k=-\infty}^{+\infty} x[k] h[n - k] \end{aligned}$$

... but let's first train some basic operations on discrete signals.

Delay (zpoždění) and advancing (předběhnutí)

$$y[n] = x[n-2]$$

n	-4	-3	-2	-1	0	1	2	3	4	5	6
x[n]					6	5	-1	3			
					2 samples delay →						
n-2	-6	-5	-4	-3	-2	-1	0	1	2	3	4
x[n-2]							6	5	-1	3	

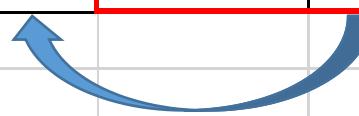
$$y[n] = x[n+3]$$

n	-4	-3	-2	-1	0	1	2	3	4	5	6
x[n]					6	5	-1	3			
					← 3 samples advance						
n+3	-1	0	1	2	3	4	5	6	7	8	9
x[n+3]		6	5	-1	3						

Flipping (otočení)

$$y[n] = x[-n]$$

n	-4	-3	-2	-1	0	1	2	3	4	5	6
x[n]					6	5	-1	3			
-n	4	3	2	1	0	-1	-2	-3	-4	-5	-6
x[n-2]		3	-1	5	6						



Flipping with a shift – counter-intuitive !!!

$$y[n] = x[-n-1]$$

n	-4	-3	-2	-1	0	1	2	3	4	5	6
x[n]					6	5	-1	3			
-n-1	3	2	1	0	-1	-2	-3	-4	-5	-6	-7
x[-n-1]	3	-1	5	6							

$$y[n] = x[-n+4]$$

n	-4	-3	-2	-1	0	1	2	3	4	5	6
x[n]					6	5	-1	3			
-n+4	8	7	6	5	4	3	2	1	0	-1	-2
x[-n+4]						3	-1	5	6		

Unsure ? Run a check !

- Select an important sample in the signal
- For the modified signal, evaluate the time modification for this sample
- Look at the resulting time in the original signal.
 - Same sample as in the modified one ? **OK**
 - Different sample ? **BAD, DO IT AGAIN**

Now finally ready for the convolution ...

$$y[n] = x[n] \star h[n] = \sum_{k=-\infty}^{+\infty} x[k]h[n-k]$$

- k is an **auxiliary variable**, usually not running from $-\infty$ to $+\infty$.
- Let's start with renaming the signals from $x[n]$ and $h[n]$ to $x[k]$ and $h[k]$ and flipping $h[k]$ to $h[-k]$

n	-4	-3	-2	-1	0	1	2	3	4	5	6
x[n]	x[-4]	x[-3]	x[-2]	x[-1]	x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
h[n]					h[0]	h[1]	h[2]	h[3]			
k	-4	-3	-2	-1	0	1	2	3	4	5	6
x[k]	x[-4]	x[-3]	x[-2]	x[-1]	x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
h[k]					h[0]	h[1]	h[2]	h[3]			
h[-k]		h[3]	h[2]	h[1]	h[0]						

- And proceed by computing output samples for different n 's

n=0											
k	-4	-3	-2	-1	0	1	2	3	4	5	6
x[k]	x[-4]	x[-3]	x[-2]	x[-1]	x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
h[0-k]		h[3]	h[2]	h[1]	h[0]						
y[n]					y[0]						
n=1											
k	-4	-3	-2	-1	0	1	2	3	4	5	6
x[k]	x[-4]	x[-3]	x[-2]	x[-1]	x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
h[1-k]			h[3]	h[2]	h[1]	h[0]					
y[n]						y[1]					
n=6											
k	-4	-3	-2	-1	0	1	2	3	4	5	6
x[k]	x[-4]	x[-3]	x[-2]	x[-1]	x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
h[6-k]								h[3]	h[2]	h[1]	h[0]
y[n]											y[6]

Convolution - summary

- Flip the impulse response
 - For all meaningful output samples n
 - Shift flipped impulse response to given „ n “
 - Mutliply
 - Add
 - Write the result to $y[n]$
- 

... „paper strip“ + Python **#convolution** for

$x = [4 \ 4 \ 4 \ 4 \ 0 \ 0 \ 0 \ 0]$

$h = [0.25 \ -0.25 \ 0.25 \ -0.25]$

Convolution is commutative

$$y[n] = x[n] \star h[n] = h[n] \star x[n] = \sum_{k=-\infty}^{+\infty} h[k]x[n-k]$$

Try it on our example with the paper-strip method as a homework
😊

Agenda

- Terminology and motivation
- Basic operations needed for filtering
- A simple FIR filter
- Convolution (and needed signal manipulations)
- **Filters with feedback - IIR**
- Frequency response - an introduction
- Summary

Recursive / IIR – infinite impulse response filters

Diference equation

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] + b_3x[n-3] \\ - a_1y[n-1] - a_2y[n-2] - a_3y[n-3]$$

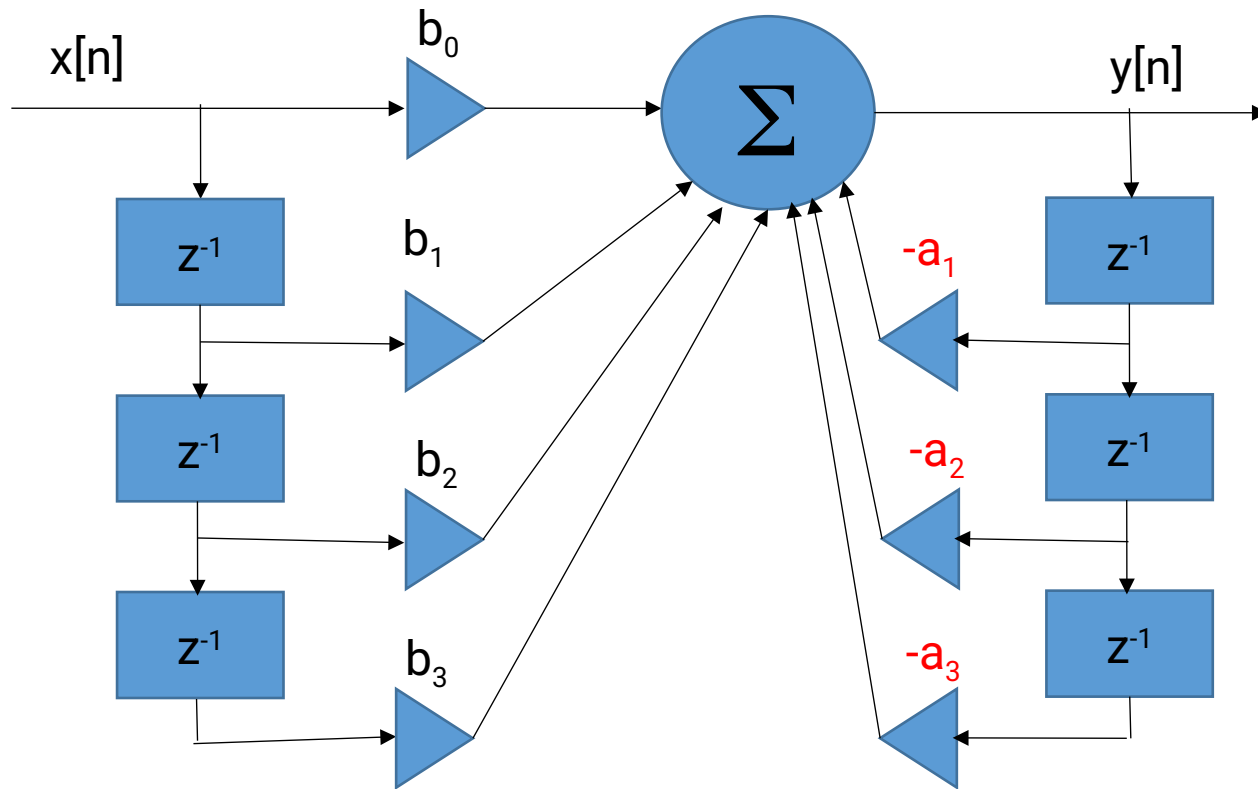
Why not a_0 ?

Can't use sample $y[n]$ as it is being made.

Why do the feed-back coefficients have negative sign ?

Will complicate things now, but make them easier later ...

Scheme



Operation – making use of filter output !

n	-4	-3	-2	-1	0	1	2	3	4	5	6
x[n]	x[-4]	x[-3]	x[-2]	x[-1]	x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
		b3	b2	b1	b0						
		Σ									
		-a3	-a2	-a1							
y[n]	y[-4]	y[-3]	y[-2]	y[-1]	y[0]	y[1]	y[2]	y[3]	y[4]	y[5]	y[6]

n	-4	-3	-2	-1	0	1	2	3	4	5	6
x[n]	x[-4]	x[-3]	x[-2]	x[-1]	x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]
								b3	b2	b1	b0
								Σ			
								-a3	-a2	-a1	
y[n]	y[-4]	y[-3]	y[-2]	y[-1]	y[0]	y[1]	y[2]	y[3]	y[4]	y[5]	y[6]

Implementation off-line ... iir_offline.c

- Digging also in the old outputs ...

```
for (n=3; n<N; n++) {  
    y[n] = b0 * x[n] + b1 * x[n-1] + b2 * x[n-2]  
        + b3 * x[n-3] - a1 * y[n-1]  
        - a2 * y[n-2] - a3 * y[n-3];  
}
```


Implementation on-line... [iir_online.c](#)

- The function has to memorize also the past outputs.

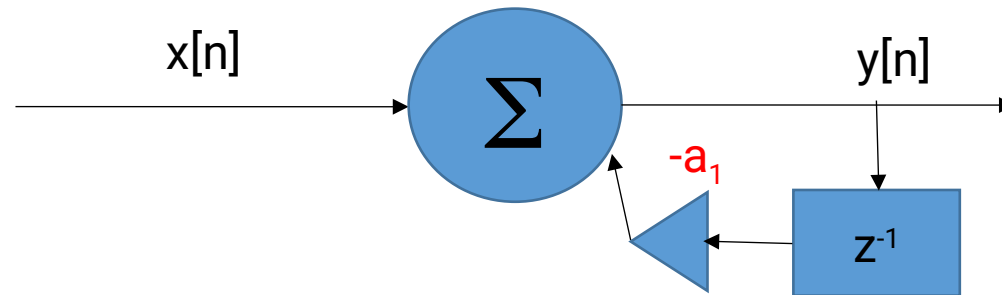
```
float filter (float xn) {  
    static float xn1 = 0.0, xn2 = 0.0, xn3 = 0.0;  
    static float yn1 = 0.0, yn2 = 0.0, yn3 = 0.0;  
    float yn;  
    yn = b0 * xn + b1 * xn1 + b2 * xn2 + b3 * xn3  
        - a1 * yn1 - a2 * yn2 - a3 * yn3;  
    xn3 = xn2; xn2 = xn1; xn1 = xn;  
    yn3 = yn2; yn2 = yn1; yn1 = yn;  
    return (yn);  
}
```

+ a call in main:

```
for (n=0; n<N; n++) {  
    y[n] = filter (x[n]);  
}
```

Infinite impulse response

- For example for a simple, purely recursive filter (nothing done with the input)



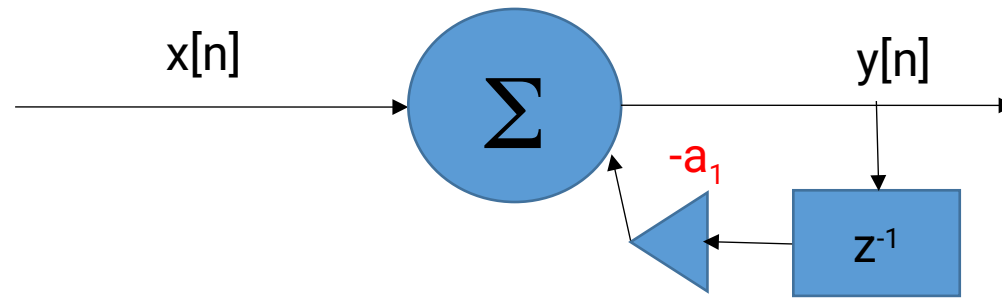
#iir_response

Stability

- Mathematically „Bounded Input Bounded Output“
- Popularly: “if reasonable stuff on the input expecting reasonable stuff on the output”
- FIR filter can not be unstable
- IIR can.

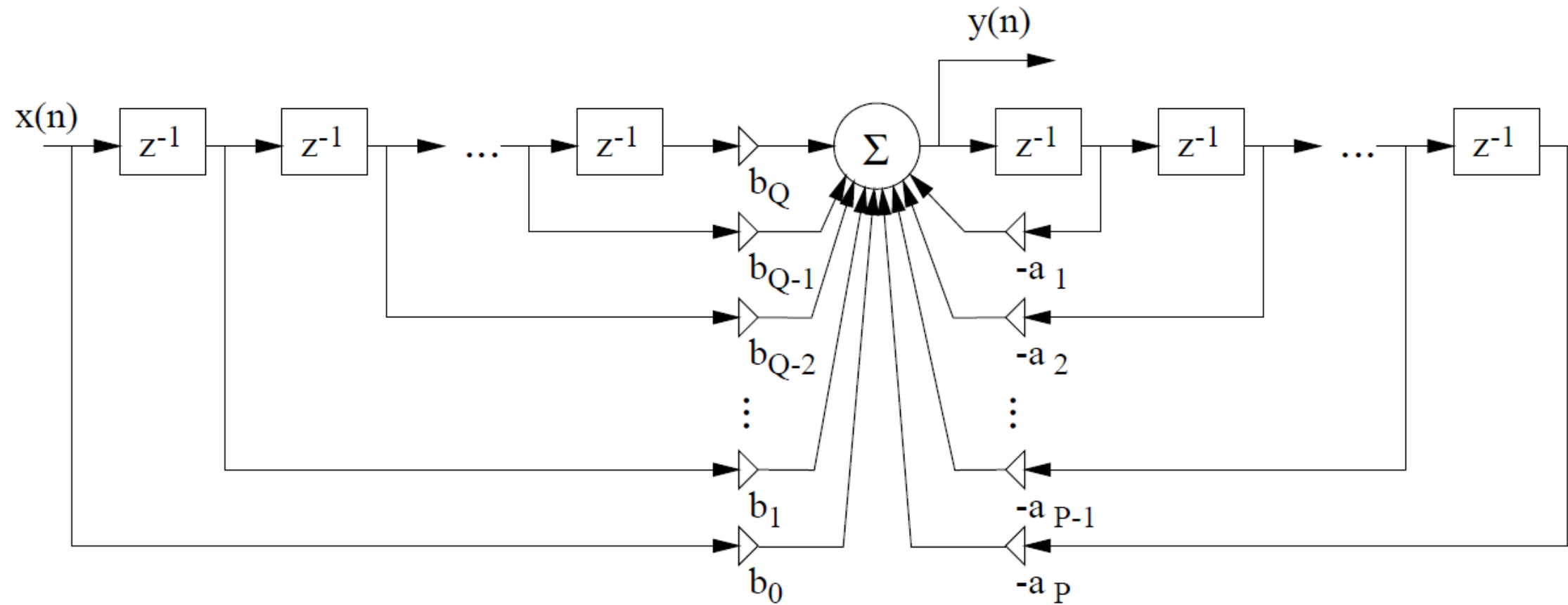


First order IIR



- What must be respected for a_1 ?
- In the following lecture, we'll see the condition of stability for more complicated filters.

General IIR filter



General filter

- Order of input (řád vstupu) Q
- **Order of output** (řád výstupu) P

Difference equation

$$y[n] = b_0x[n] + b_1x[n-1] + \dots + b_Qx[n-Q] \\ - a_1y[n-1] + \dots + a_Py[n-P]$$

$$y[n] = \sum_{k=0}^Q b_k x[n-k] - \sum_{k=1}^P a_k y[n-k]$$

Implementation off-line `iirbig_offline.c`

- ... still the same, this time with cycles

```
for (n=P; n<N; n++) {  
    sum = 0.0; /* zero the accumulator */  
    for (k=0; k<=Q; k++) {  
        sum += B[k] * x[n-k];  
    }  
    for (k=1; k<=P; k++) {  
        sum -= A[k] * y[n-k];  
    }  
    y[n] = (int16_t)sum;  
}
```

Implementation on-line **iirbig_online.c**

```
float filter (float xn) {  
    definition of static double xbuf (x[n] ... x[n-Q-1]) and ybuf (y[n] ... y[n-P-1]) ...  
    yn = 0.0;  
    xbuf[0] = xn;  
    for (k = Q; k >= 0; k--) {  
        yn += xbuf[k] * B[k];  
        xbuf[k+1] = xbuf[k];  
    }  
    for (k = P; k >= 1; k--) {  
        yn -= ybuf[k] * A[k];  
        ybuf[k+1] = ybuf[k];  
    }  
    ybuf[1] = yn;  
    return yn;  
}
```


Implementation on-line - notes

- The cycles must run backward (otherwise the old values rewrite the new ones)
- Each body of cycle performs multiplication, addition and delay.
- The cycle for the **output** must stop at index 1 (coefficient a_0 does not exist !)
- „Trash“ memory fields `xbuf[Q+1]` , `ybuf[P+1]` , `ybuf[0]` will never be used – makes indexing easier and save us some „if“s.

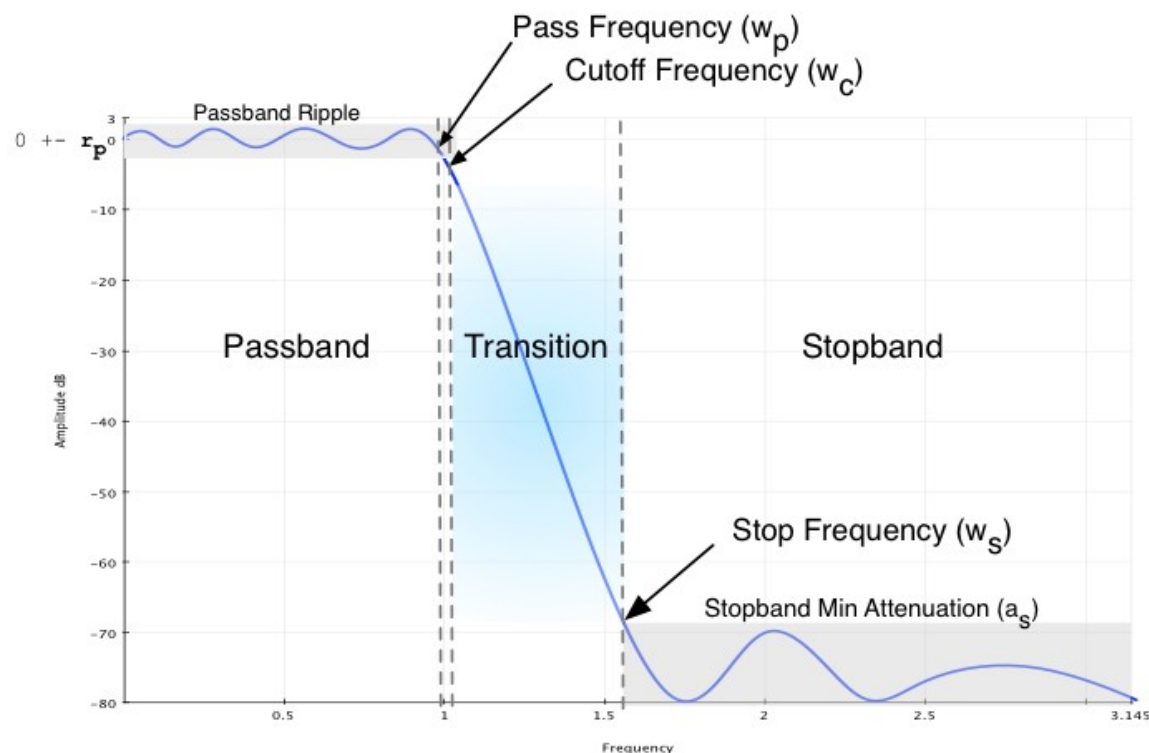
Agenda

- Terminology and motivation
- Basic operations needed for filtering
- A simple FIR filter
- Convolution (and needed signal manipulations)
- Filters with feedback - IIR
- **Frequency response - an introduction**
- Summary

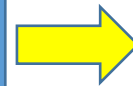
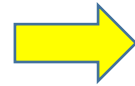
Representation of filter in frequency

Frequency response (frekvenční / kmitočtová charakteristika)

- Interested, how the filter behaves in frequency
 - i.e. how individual frequencies are amplified / attenuated
 - and how they are shifted
- Determining the type of filter
 - Low-pass, High-pass, Band-pass, Band-stop
- Filter design process - specifications
 - pass-band should be from xx Hz to yy Hz
 - stop-band should be from zz Hz to ww Hz
 - attenuation should be minimum aa dB
 - pass-band ripple max. bb dB.
- We need the frequency response to check whether the design worked.



Frequency response historically – measure !



Frequency response from impulse response

- FIR filters:
 - Generate impulse response
 - Perform its frequency transform – DFT, DTFT
 - Works, as the impulse response is **finite**.

#freq_char_by_ft_of_impulse_response
- IIR filters
 - Can not get the impulse response precisely, as it is **infinite**.
 - Can truncate it and run DFT / DTFT, but the result will not be correct
 - We have better ways to do it – wait for the next lecture 😊

Agenda

- Terminology and motivation
- Basic operations needed for filtering
- A simple FIR filter
- Convolution (and needed signal manipulations)
- Filters with feedback - IIR
- Frequency response - an introduction
- **Summary**

Summary

Describing the filter by

- Scheme
- Difference equation
- Impulse response
- Transfer function
- Frequency response

Summary II.

- Types of filters
 - FIR
 - IIR
- Implementation
 - Multiplication
 - Summation
 - Shifts
 - Off-line
 - On-line

Summary III.

- Convolution
 - Way to obtain the output of FIR filter for an input signal.
 - Can be easily imagined and related to the difference equation.
 - Try it out with the paper strips !
- Computing the frequency response
 - Ugly:
 - „measurement“
 - Analysis of $h[n]$
 - Nicely - using z-transform – TBD next time